COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithms	**SECTION:**	CPE21S4
DATE PERFORMED:	9/4/2026	**DATE SUBMITTED:**	9/4/2026
NAME:	Peñaflorida,Wrio Carlo	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. ___ [ACTIVITY NAME]

1. PROCEDURE OUTPUT

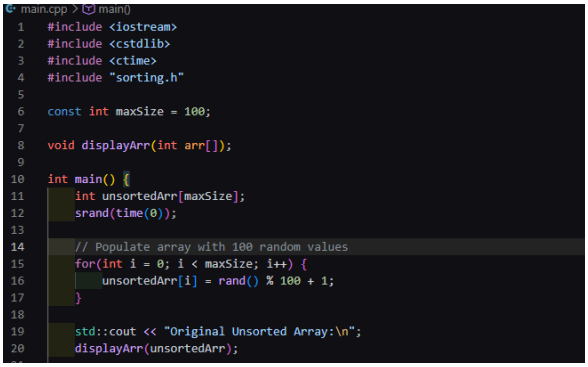
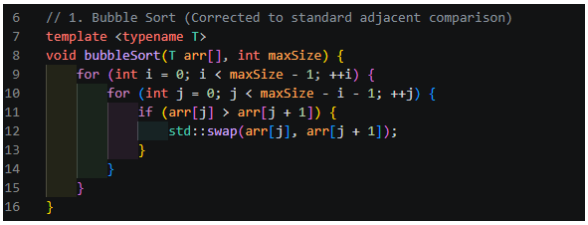
Code + Console Screenshot	
Observations	The array is successfully populated with 100 pseudo-random integers. Using srand(time(0)) ensures the sequence is seeded by the current system time, resulting in a completely different, unsorted list every time the program is executed.

Table 7-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot	
Observations	The algorithm successfully sorts the array by repeatedly comparing adjacent elements and swapping them if the left element is greater than the right. During each pass, the largest remaining unsorted element "bubbles" to its correct position at

	the end of the array. It performs exactly as expected but requires many comparisons and swaps, demonstrating its $O(n^2)$ time complexity.

Table 7-2. Bubble Sort Technique

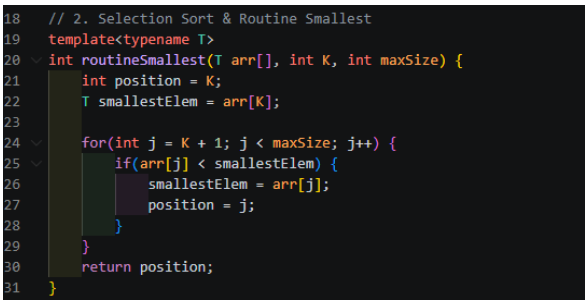
Code + Console Screenshot	
Observations	The algorithm effectively sorts the data by searching the unsorted portion of the array for the absolute minimum value and swapping it with the first unsorted position. It visibly executes fewer swaps overall compared to Bubble Sort, though it still must scan the remaining elements on every pass, maintaining an $O(n^2)$ complexity.

Table 7-3. Selection Sort Algorithm

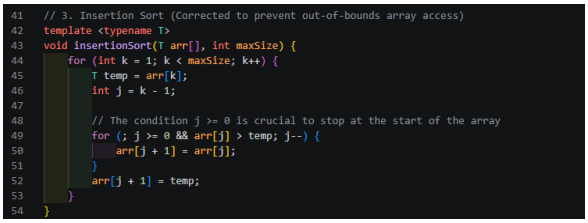
Code + Console Screenshot	
Observations	The array is sorted by taking one element at a time and shifting previous elements to the right until the correct position is found for insertion. It correctly handles the bounds of the array. While it shares an $O(n^2)$ worst-case time complexity with the previous two algorithms, it is generally faster in practice for small datasets because it stops scanning once the correct insertion point is found.

Table 7-4. Insertion Sort Algorithm

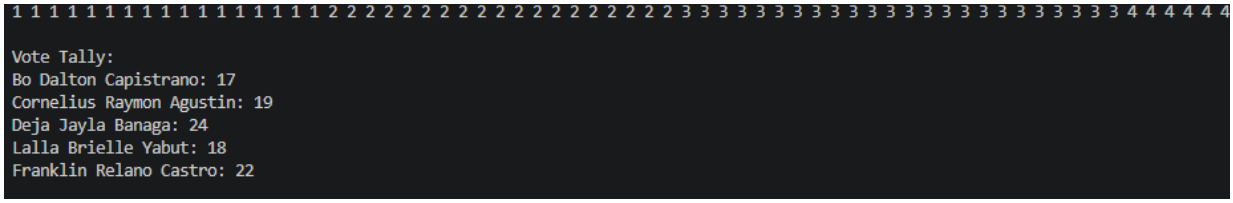
FINAL OUPUT:



2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given data sorting problems using appropriate basic sorting algorithms
Candidate 1 Bo Dalton Capistrano
Candidate 2 Cornelius Raymon Agustín
Candidate 3 Deja Jayla Bañaga
Candidate 4 Lalla Brielle Yabut
Candidate 5 Franklin Relano Castro
List of Candidates
Problem: Generate an array A[0…100] of unsorted elements, wherein the values in the array are indicative of a vote to a candidate. This means that the values in your array must only range from 1 to 5. Using sorting and searching techniques, develop an algorithm that will count the votes and indicate the winning candidate.
NOTE: The sorting techniques you have the option of using in this activity can be either bubble, selection, or insertion sort. Justify why you chose to use this sorting algorithm.
Deliverables:
Pseudocode of Algorithm
Screenshot of Algorithm Code
Output Testing

Output Console Showing Sorted Array	Manual Count	Count Result of Algorithm
	21	21
	18	18
	24	24



Conclusion:

The practical application of sorting algorithms greatly simplifies data aggregation; by sorting the array first, disorganized voting data groups naturally into contiguous blocks, mimicking a manual ballot-sorting process. Randomizing the array to simulate this dataset ensured the testing covered diverse scenarios, while utilizing the external sorting header compartmentalized the logic to keep the main file focused strictly on the problem's tallying requirements. Restricting the candidate values between 1 and 5 reinforced how array indexing serves as an effective frequency counter, allowing the program to combine searching and tallying into a single efficient pass. Ultimately, the activity was successfully completed with a streamlined logic flow, with the primary area for improvement in future iterations being the addition of a tie-breaker mechanism for the final vote count. array to simulate a dataset ensures testing covers diverse scenarios. The integration of sorting.h compartmentalized the logic smoothly, keeping main.cpp focused strictly on the problem requirements.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.